TecnoCampus

Centre universitari adscrit a la

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 2

Web Application Vulnerabilities

Student Assignment

Course	Introduction to Cybersecurity
Lab	2 of 4
Topic	Stored XSS & Session Hijacking
Duration	90–120 minutes
Difficulty	Intermediate
Flags	3

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Learning Objectives

After completing this lab you will be able to:

- Identify and exploit a **Stored Cross-Site Scripting (XSS)** vulnerability in a web application
- Understand why cookies without the `HttpOnly` flag are dangerous
- Exfiltrate a session cookie from an authenticated victim using a malicious payload
- Perform **session hijacking** by replaying a stolen cookie in a different browser
- Reflect on the mitigation measures that would prevent each step of the attack

Scenario

You are a penetration tester assessing **Synapse Corp** – a company that runs an internal web platform for employee communications. The application includes a public comments section that any visitor can read and post to.

During your recon you noticed that no input validation seems to be applied to comments. A simulated employee (represented by an automated browser) is known to log in periodically and visit the comments page.

Your objectives are:

1. Confirm the comments section is vulnerable to Stored XSS.
2. Use that vulnerability to steal the authenticated employee's session cookie.
3. Use the stolen cookie to access the application as that employee without knowing their password.

Rules of engagement

- Only interact with the lab environment (`192.168.30.10`)
- Do not attempt to modify the Docker containers or the server infrastructure
- Document every payload you try, even if it does not work

- If the listener stops receiving requests, re-read the victim's cycle time and try again

Network Topology

```
[ Attacker - Parrot OS ]   (10.0.40.5)
          |
          | (EVE-NG management network)
          |
  [ nginx proxy ]  <-- entry point: http://192.168.30.10
          |
  [ flask app ]    (vulnerable Synapse application)
          |
  [ victim ]       (Playwright browser -- simulated authent
```

Node	Role	Access
Attacker	Your machine (Parrot OS)	VNC console
nginx	Reverse proxy exposing the web app	http://192.168.30.10
flask	Vulnerable Python web application (Synapse)	Via nginx
victim	Automated Playwright browser (simulated user)	Internal only

Table 1: Lab nodes overview

Entry point: The application is reachable at `http://192.168.30.10`. The victim container authenticates as user `guest` and visits `/comments` every ~20 seconds.

Tasks

Complete the following tasks in order. Document every command, payload, and screenshot.

Task 1 – Application Reconnaissance

1. Browse to `http://192.168.30.10` and explore the application.

2. Identify the routes that are publicly accessible without authentication.
3. Navigate to `/comments` and read the existing entries.
4. Inspect the page source and identify where user input is rendered.

Deliverable: Which routes did you find? Is the input reflected raw in the HTML?

Task 2 – Stored XSS Verification (Flag 1)

1. Post a comment containing a classic XSS proof-of-concept payload.
2. Log in as `guest` (credentials on the login page hint) and revisit `/comments` to trigger execution in a legitimate session.
3. Confirm that JavaScript executes in the context of an authenticated user.

Deliverable: Screenshot of the XSS executing. Flag 1 appears inside the alert box – read it carefully.

Task 3 – Cookie Exfiltration

1. On your Parrot machine, start an HTTP listener on port `8000`.
2. Craft a payload that sends `document.cookie` to your listener as a query parameter.
3. Post the payload as a comment.
4. Wait for the victim container to visit `/comments` and observe the incoming request on your listener.
5. Decode the URL-encoded cookie value from the log line.

Deliverable: The full cookie value received on your listener.

Task 4 – Session Hijacking (Flag 2)

1. Open a private/incognito browser window and navigate to `http://192.168`
2. Without entering any credentials, inject the stolen cookie into the browser.
3. Reload the page and verify you are recognised as `guest`.
4. Navigate to a route that requires authentication and capture Flag 2.

Deliverable: Flag 2 string. Explain in one sentence why this works.

Deliverables

Submit a report (1–2 pages) covering:

- **XSS confirmation:** payload used and screenshot of execution
- **Cookie exfiltration:** listener output with the raw request line
- **Session hijacking:** how you injected the cookie and the result
- **Both flags captured:** exact flag strings
- **Reflection:** at least two specific mitigations that would break this attack chain

Hints

Stuck? Hints are ordered by how much they reveal.

1. The comments section renders input directly in the HTML without escaping. Think about what a browser does when it encounters a `<script>` tag.
2. A minimal XSS payload to confirm execution: `<script>alert(1)</scri`
3. To receive the cookie, you need a server that logs HTTP requests. `python3 -m http.server 8000` prints every GET request to stdout.
4. Your listener IP inside the lab is `10.0.40.5`. The victim's browser must be able to reach it directly.
5. To inject a cookie from the browser console: `document.cookie = "name=value; path=/";`
6. The victim visits `/comments` roughly every 20 seconds. Be patient – it may take up to one full cycle after posting the payload.

Tools Reference

The following tools are available on the Attacker node:

Tool	Use
`firefox`	Browser for manual testing and cookie injection
`curl`	Command-line HTTP requests
`python3 -m http.server`	Minimal HTTP listener to capture exfiltrated data
Developer Tools	Inspect cookies (Storage/Application tab) and console
`burpsuite`	Optional – intercept and replay HTTP requests

Table 2: Available tools on Attacker node